Teamsync – Software Requirements Specification

Nov 11, 2026 · @Fahim

1. Introduction

Teamsync is a web application where a student team plans, assigns and tracks its project tasks on one shared board, and the professor follows every team's progress. This is version 1.0 of its requirements, written after Sprint 1.

1.1 Purpose

This Software Requirements Specification (SRS) states what Teamsync must do (functional requirements), how well it must do it (non-functional requirements), and how each requirement is traced to a use case, a user story, a test and a GitHub issue. Its readers are the course professor and the four team members.

1.2 Scope

In scope: accounts with roles, projects with invite codes, a Kanban board with tasks, deadline reminders, a contribution report per member, and a read-only professor dashboard.

Out of scope for version 1: password reset, email verification, file attachments, chat, mobile apps and online hosting.

1.3 Definitions

Term	Meaning
Project	One team's workspace, with a title, a leader and members
Invite code	A 7-character code (for example TS-KKK7) that lets a student join a project
Task	A piece of work with a title, assignee, priority, due date and status
Board	The Kanban view of a project: To Do, In Progress and Done columns

Term	Meaning
Sprint	A one-week Scrum iteration that ends with working, merged software
FR / NFR	Functional / non-functional requirement

1.4 Team and course

Software Engineering, Dong-eui University, Fall 2026. Code and backlog: Teamsync on GitHub.

Name	Student ID	Role
Kabir Md Fahim	20244154	Team Leader, Scrum Master
Miraj Md	20244166	Team Member
Fahim Iftekhar Zaman	20244229	Team Member
Hossain Md Fahim Al	20244220	Team Member

2. Software process

Teamsync follows **Agile (Scrum) with a light up-front requirements and design phase**: one-week sprints, a product backlog of user stories, and working software merged at the end of every sprint. As Sommerville notes, most real projects mix plan-driven and agile practice; we plan just enough (this SRS, UML and a database design) and build the rest in sprints.

2.1 Why Scrum and not Waterfall

- **Fixed, short deadline:** about five weeks to the final demo. Scrum delivers a working part every week; Waterfall delivers nothing usable until the end.
- **Requirements change:** the professor is our customer, and feedback (such as asking for this document) arrives during the project. Scrum adds it to the backlog for the next sprint.
- **Testing every sprint:** each story is tested against its acceptance criteria before it is merged, instead of one test phase at the end with no time left to fix bugs.
- **Small team:** Scrum is built for small teams of 3 to 9 people; we are 4.

2.2 How we run Scrum

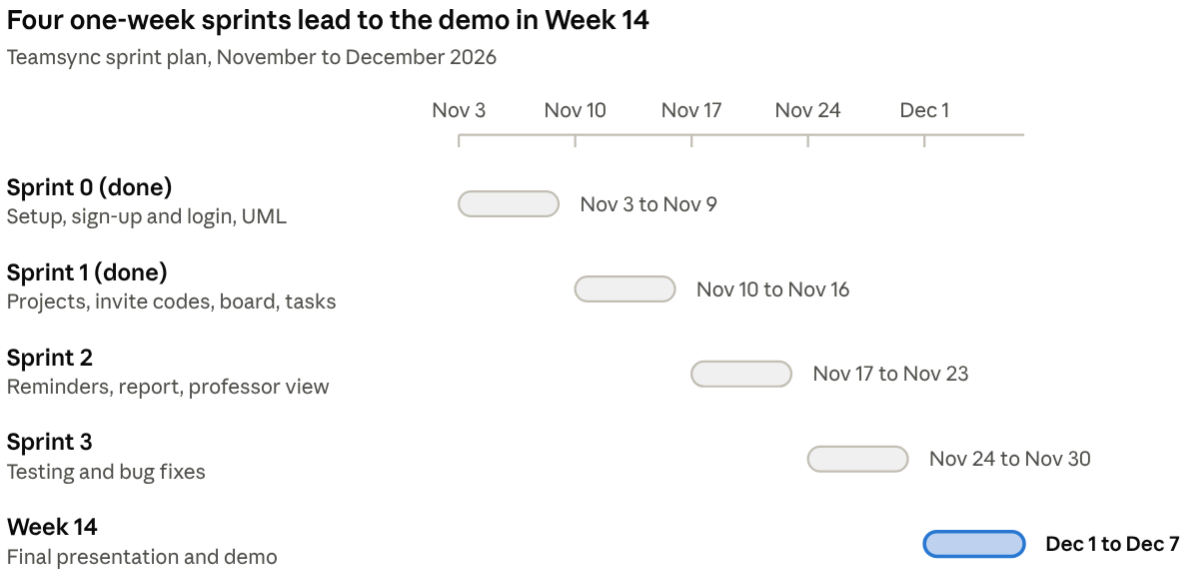
Practice	How we do it	Evidence
Product backlog	Every user story is a GitHub Issue	Issues #1 to #4 and later
Sprint	One week, Monday to Sunday	Teamsync Sprint Board on GitHub Projects
Sprint planning	At the start of each sprint, stories are picked and assigned	Issue assignees and the To Do column
Daily stand-up	Three lines in the team chat: done yesterday, doing today, blocked by	Team chat history
Sprint review	Demo of the merged features at the end of the sprint	Merged Pull Requests
Retrospective	What to keep and what to change, noted after each review	Notes in the docs folder

Roles. Scrum Master and Product Owner: Kabir Md Fahim. Customer: the course professor. Development team: the four team members, each with the responsibility below.

Member	Responsibility	Where it shows
Kabir Md Fahim	Scrum Master and Product Owner; development (PHP, MySQL, front end); reviews and merges Pull Requests	Commits and Pull Requests
Miraj Md	Keeps the UML diagrams (use case, class, sequence) and the ER diagram up to date each sprint	`docs/` folder
Fahim Iftekhar Zaman	Writes the test cases in section 9 and runs them each sprint before a merge	`tests/` folder
Hossain Md Fahim Al	Final presentation slides and the user guide	`docs/` folder

Definition of Done. A story is done when its code is merged into `main` through a Pull Request that says `closes #n`, every acceptance criterion has been checked on XAMPP, and the GitHub issue has closed and moved to Done.

2.3 Sprint plan



sprint plan · four sprints and the demo week

Sprint 1 finished early: all four of its stories were merged on Nov 10, its first day.

3. Stakeholders and user roles

Teamsync has three user roles, chosen at sign-up; what each can do is enforced on the server, not only by hiding buttons.

Role	Who	Goal	Can do	Cannot do
Team Leader	The student who runs the team	Split the work fairly and see progress	Create a project, add, edit, delete and move any task, join other projects	Change another project's tasks
Team Member	Every other student in the team	Know their own tasks and deadlines	Join with an invite code, see the board, move tasks assigned to them	Add, edit or delete tasks, move others' tasks
Professor	The course instructor	See every team's progress and each member's contribution	View all projects, boards and contribution reports (Sprint 2)	Create, edit, move or delete anything

The professor is also the project's **customer**: the person whose feedback sets the requirements. The team leader is a role inside one project; a leader of one team can be a member of another.

4. Functional requirements

Teamsync has 24 functional requirements; the 16 for accounts, projects and the board are done, and the 8 for drag-and-drop, reminders, reports and the professor view are planned for Sprint 2. Priority uses MoSCoW: Must, Should, Could.

ID	Requirement	Priority	Sprint	Status
FR-01	The system shall let a visitor create an account with name, email, a password of at least 8 characters, and a role (Team Leader, Team Member or Professor).	Must	0	Done
FR-02	The system shall refuse a sign-up whose email is already registered.	Must	0	Done
FR-03	The system shall let a registered user log in with email and password, and log out.	Must	0	Done
FR-04	The system shall open each page only for the roles allowed to use it.	Must	0	Done
FR-05	The system shall let a Team Leader create a project with a title of 1 to 100 characters and give it a unique 7-character invite code (TS-XXXX).	Must	1	Done
FR-06	The system shall make the creator the project's leader and first member.	Must	1	Done
FR-07	The system shall let a Team Leader or Team Member join a project by typing its invite code, ignoring letter case, spaces and dashes.	Must	1	Done
FR-08	The system shall show "Invite code not found" for a wrong code and "You are already in this project" for a repeat join.	Must	1	Done
FR-09	The system shall list on the dashboard every project the user belongs to, with its invite code.	Must	1	Done
FR-10	The system shall show a project's tasks on a Kanban board with To Do, In Progress and Done columns.	Must	1	Done

ID	Requirement	Priority	Sprint	Status
FR-11	The system shall show on each card the title, priority, assignee and due date, and mark a task past its due date as Overdue.	Must	1	Done
FR-12	The system shall show the project's progress as done tasks out of all tasks, in percent.	Should	1	Done
FR-13	The system shall let the project's leader add a task with title, assignee (a project member), priority and optional due date; a new task starts in To Do.	Must	1	Done
FR-14	The system shall let the project's leader edit and delete tasks, and ask for confirmation before a delete.	Must	1	Done
FR-15	The system shall let the leader move any task, and a member move only tasks assigned to them, between columns.	Must	1	Done
FR-16	The system shall show a project's board only to its members (and read-only to professors, FR-24).	Must	1	Done
FR-17	The system shall let users move cards by drag-and-drop as well as with buttons.	Could	2	Planned
FR-18	The system shall show each user, on the dashboard, their tasks that are overdue or due within 2 days.	Should	2	Planned
FR-19	The system shall email the assignee one day before a task's due date.	Could	2	Planned
FR-20	The system shall show a contribution report per project: tasks done, in progress and to do for each member.	Must	2	Planned
FR-21	The system shall draw the contribution report as a bar chart.	Should	2	Planned
FR-22	The system shall show a professor a dashboard of all projects with member count, progress and number of overdue tasks.	Must	2	Planned
FR-23	The system shall refuse every create, edit, move and delete request from a professor.	Must	2	Planned
FR-24	The system shall let a professor open any project's board and contribution report, read-only.	Should	2	Planned

5. Non-functional requirements

Every non-functional requirement below has a number or a pass/fail test, so it can be verified rather than argued about. Types follow Sommerville: product, organisational and external requirements.

ID	Type	Requirement	How we check it
NFR-01	Product: performance	The board page loads in 2 seconds or less with 100 tasks, on XAMPP on a student laptop.	Browser developer tools, Network tab
NFR-02	Product: performance	Moving a card saves and shows the updated board in 1 second or less.	Browser developer tools, Network tab
NFR-03	Product: security	Passwords are stored only as bcrypt hashes (`password_hash`), never as plain text.	Look at the `users` table: every value starts with `$2y$`
NFR-04	Product: security	Every SQL query uses a PDO prepared statement; input such as `' OR 1=1 --` changes nothing.	Code review; injection test on the join form
NFR-05	Product: security	Every form that changes data carries a CSRF token; a request without it is refused with HTTP 400.	Send a request without the token
NFR-06	Product: security	All user text is escaped before it is shown; a task titled `<script>` appears as text.	XSS test with a script tag in a title
NFR-07	Product: security	Permissions are checked on the server: opening a page or API URL without the right role or membership returns HTTP 403.	Open another project's board by its URL

ID	Type	Requirement	How we check it
NFR-08	Product: usability	A new user can sign up, join a project and move a task in 3 minutes or less without help.	Timed test with 3 classmates
NFR-09	Product: usability	Buttons are at least 40 px tall, every form field has a label, and text contrast is at least 4.5:1.	Lighthouse accessibility score of 90 or more
NFR-10	Product: compatibility	Works in current Chrome, Safari and Edge, and at a 390 px phone width, where the board columns stack.	Manual test on each browser and a phone
NFR-11	Product: reliability	Creating a project and adding its leader as a member happen in one transaction: both are saved or neither is.	Code review of `create_project()`
NFR-12	Organisational: development	Every change reaches `main` through a Pull Request that closes its GitHub issue.	GitHub history
NFR-13	Organisational: environment	Runs on XAMPP (PHP 8, MySQL or MariaDB) on Windows and macOS; only `config/db.php` differs per computer.	Set up on a second computer from the README
NFR-14	External: privacy	Stores only name, email, role and project data; no student ID or phone number, and passwords are never shown.	Review of `database/schema.sql`

6. User stories and acceptance criteria

The product backlog holds 9 user stories: 5 are done (Sprint 0 and Sprint 1) and 4 are planned for Sprint 2. A story is accepted only when every criterion under it passes.

Sprint 0: setup (done)

US-01 · Sign up and log in · FR-01 to FR-04 · starter commit

As a student, I want to sign up and log in with my email, so that only my team can see our work.

- Sign-up asks for name, email, a password of 8 or more characters, and a role
- An email already in use shows "An account with this email already exists"
- A wrong password shows "Wrong email or password"
- Log out ends the session and returns to the login page

Sprint 1: projects and board (done)

US-02 · Create a project · FR-05, FR-06 · issue #1

As a team leader, I want to create a project and get an invite code, so that my teammates can join it.

- The leader enters a project title and clicks Create
- The system generates a unique 7-character code (for example TS-4821)
- The project is saved and the leader is added as its first member
- The invite code is shown on the dashboard

US-03 · Join a project · FR-07 to FR-09 · issue #2

As a team member, I want to join a project with an invite code, so that I can see my team's tasks.

- The member enters a code and clicks Join
- A wrong code shows "Invite code not found"
- Joining the same project twice shows "You are already in this project"
- The member is added to `project_members` and sees the project on the dashboard

US-04 · Kanban board · FR-10 to FR-12, FR-15, FR-16 · issue #3

As a team member, I want to see all tasks in To Do, In Progress and Done columns, so that I know what the team is working on.

- The board shows three columns with the tasks of the selected project
- Each card shows title, assignee, priority and due date
- Only members of the project can open its board
- The board works on a phone screen, with the columns stacked

US-05 · Add and edit tasks · FR-13, FR-14 · issue #4

As a team leader, I want to add and edit tasks with an assignee, deadline and priority, so that everyone knows their work.

- The form has title, assignee, priority and due date
- The title is required, at most 100 characters
- A new task appears in the To Do column
- Only the leader can edit or delete a task

Sprint 2: reminders, report and professor view (planned)

US-06 · Deadline reminders · FR-18, FR-19 · issue to be created

As a team member, I want to see my late and soon-due tasks when I log in, so that I don't miss a deadline.

- The dashboard lists the user's unfinished tasks that are overdue or due within 2 days
- Overdue tasks come first and are marked "Overdue"
- Only the user's own tasks appear
- With nothing due, it says "No deadlines in the next 2 days"

US-07 · Contribution report · FR-20, FR-21 · issue to be created

As a team leader, I want to see how many tasks each member has finished, so that the work is shared fairly.

- One row per member: tasks done, in progress and to do
- A bar chart shows the same numbers
- Only project members and professors can open the report
- Unassigned tasks are counted on their own row

US-08 · Professor dashboard · FR-22 to FR-24 · issue to be created

As a professor, I want to see every team's progress on one page, so that I can spot teams that are falling behind.

- One row per project: title, members, progress and overdue tasks
- Each row opens that project's board and report, read-only
- The professor sees no add, edit, move or delete buttons
- Any change request from a professor account is refused with HTTP 403

US-09 · Drag-and-drop · FR-17 · issue to be created

As a team member, I want to drag a card to another column, so that updating my work takes one move.

- Dropping a card on a column saves its new status

- Only cards the user is allowed to move can be dragged
- The progress bar updates without reloading the page
- The Start, Done and Back buttons still work without JavaScript

7. Use case descriptions

These describe the 8 use cases in the Use Case diagram. UC-01 to UC-05 are built; UC-06 to UC-08 are planned for Sprint 2.

UC-01 Log in

Actor: any user · **Related:** FR-03, US-01 · **Precondition:** the user has an account.

1. The user opens Teamsync and enters email and password.
2. The system checks the password against the stored hash.
3. The system starts a new session and opens the dashboard.

Alternative flow: 2a. Email or password is wrong: the page shows "Wrong email or password." and the user stays logged out.

Postcondition: the user is logged in with their role.

UC-02 Create project

Actor: Team Leader · **Related:** FR-05, FR-06, US-02 · **Precondition:** the leader is logged in.

1. The leader clicks **+ Create project** on the dashboard.
2. The leader types a title and clicks **Create project**.
3. The system checks the title (1 to 100 characters).
4. The system generates a unique invite code, then saves the project and the leader as its member in one transaction.
5. The dashboard shows the new invite code.

Alternative flows: 3a. The title is empty or too long: the form shows "Enter a project title (up to 100 characters)." 4a. The database fails: nothing is saved and the form shows "Could not create the project. Please try again."

Postcondition: the project exists with the leader as its first member.

UC-03 Join project with an invite code

Actor: Team Member or Team Leader · **Related:** FR-07, FR-08, US-03 · **Precondition:** the user is logged in and has a code from a leader.

1. The user clicks **Join project** and types the code.

2. The system cleans the code (upper case, spaces and dashes removed) and finds the project.
3. The system adds the user to the project's members.
4. The dashboard shows "You joined" and the project.

Alternative flows: 2a. No project has the code: "Invite code not found." 3a. The user is already a member: "You are already in this project."

Postcondition: the user is a member and can open the project's board.

UC-04 Assign tasks and deadlines

Actor: the project's Team Leader · **Related:** FR-13, FR-14, US-05 · **Precondition:** the leader is on the project's board.

1. The leader clicks **+ Add task**.
2. The leader enters the title, picks an assignee from the members, a priority and an optional due date.
3. The system checks the input and saves the task in To Do.
4. The board shows the new card and "Task added to To Do."

Alternative flows: 3a. Missing or too long title, an assignee who is not a member, or an invalid date: the form lists each problem and keeps what was typed. 1a. A user who is not the project's leader opens the form: HTTP 403.

Postcondition: the task is on the board, assigned and dated.

UC-05 Update task status

Actor: the task's assignee or the project's leader · **Related:** FR-15, US-04 · **Precondition:** the user is on the board.

1. The user clicks **Start →**, **Done →** or **← Back** on a card.
2. The system checks that the user is a member and may move this task.
3. The system saves the new status and reloads the board with the new progress.

Alternative flow: 2a. The user may not move this task: "Only the person assigned to this task or the team leader can move it." (HTTP 403).

Postcondition: the task is in its new column; the sequence diagram shows this flow.

UC-06 Receive deadline reminders (planned)

Actor: Team Member or Team Leader · **Related:** FR-18, FR-19, US-06. On login, the dashboard lists the user's overdue tasks and tasks due within 2 days, overdue first.

UC-07 View contribution report (planned)

Actor: Team Leader, Team Member or Professor · **Related:** FR-20, FR-21, US-07. The report shows tasks done, in progress and to do for each member, as a table and a bar chart.

UC-08 View all teams' progress (planned)

Actor: Professor · **Related:** FR-22 to FR-24, US-08. The professor sees one row per project with members, progress and overdue tasks, and opens any board or report read-only.

8. Constraints and assumptions

Teamsync is built with plain web technology on XAMPP, by a four-student team, for a demo in Week 14 (Dec 1 to 7, 2026).

Constraints

- **Technology:** HTML, CSS and JavaScript in the browser; PHP 8 on the server; MySQL or MariaDB; no frameworks. Each team member runs it locally on XAMPP.
- **Time:** about five weeks of part-time work, planned as four one-week sprints.
- **Hosting:** version 1 runs on the team's laptops for development and the demo; it is not hosted online, so each computer has its own database.
- **Time zone:** all dates and deadlines use Korea time (Asia/Seoul).

Assumptions

- The system serves one course, so a professor account sees every project.
- Every user has a working email address, but Teamsync does not verify it.
- A student is honest about the role they pick at sign-up; there is no admin approval of roles.
- Teams have 3 to 6 members and fewer than 200 tasks per project.

9. Requirements traceability

Every functional requirement traces to a use case, a user story, a test case and a GitHub issue; planned requirements get their issues at Sprint 2 planning.

FR	Use case	User story	Test case	Issue
FR-01	UC-01	US-01	TC-01: sign up with valid data shows "Account created"	Starter commit

FR	Use case	User story	Test case	Issue
FR-02	UC-01	US-01	TC-02: sign up with a used email shows the error	Starter commit
FR-03	UC-01	US-01	TC-03: a wrong password is refused; log out returns to login	Starter commit
FR-04	UC-01	US-01	TC-04: a member opening the create-project page gets HTTP 403	Starter commit
FR-05	UC-02	US-02	TC-05: a new project shows a TS-XXXX code; 2,000 generated codes are all unique	#1
FR-06	UC-02	US-02	TC-06: after creating, the leader is listed as a member with the Leader tag	#1
FR-07	UC-03	US-03	TC-07: joining with "ts kkk7" (lower case, space) works	#2
FR-08	UC-03	US-03	TC-08: a wrong code and a repeat join show their messages	#2
FR-09	UC-03	US-03	TC-09: the dashboard lists the joined project and its code	#2
FR-10	UC-05	US-04	TC-10: the board shows three columns with the right counts	#3
FR-11	UC-05	US-04	TC-11: a task past its due date shows "Overdue"	#3
FR-12	UC-05	US-04	TC-12: 1 of 4 tasks done shows 25%	#3
FR-13	UC-04	US-05	TC-13: a new task appears in To Do; an empty title is refused	#4
FR-14	UC-04	US-05	TC-14: edits are saved; delete asks first; a member gets HTTP 403	#4
FR-15	UC-05	US-04	TC-15: a member moves their own task; moving another's gets HTTP 403	#3
FR-16	UC-05	US-04	TC-16: a non-member opening the board URL gets HTTP 403	#3
FR-17	UC-05	US-09	TC-17: dragging a card to Done saves it without a reload	Sprint 2

FR	Use case	User story	Test case	Issue
FR-18	UC-06	US-06	TC-18: a task due tomorrow appears in the due-soon list	Sprint 2
FR-19	UC-06	US-06	TC-19: the assignee gets an email one day before the due date	Sprint 2
FR-20	UC-07	US-07	TC-20: per-member counts match the board	Sprint 2
FR-21	UC-07	US-07	TC-21: the chart's bars match the table	Sprint 2
FR-22	UC-08	US-08	TC-22: a professor sees every project with its progress	Sprint 2
FR-23	UC-08	US-08	TC-23: a professor's move request gets HTTP 403	Sprint 2
FR-24	UC-08	US-08	TC-24: a professor opens a board and sees no edit buttons	Sprint 2

10. Requirements validation and change management

Requirements are validated with the professor at reviews, through UI prototypes, and by writing a test for every requirement; any change goes through a GitHub issue and a new version of this document.

10.1 Validation checks (Sommerville)

Check	Question we ask	How we check it
Validity	Does it meet a real need of a leader, member or professor?	Requirements review with the professor; each user story names who benefits
Consistency	Do any two requirements conflict?	Review of FR-16 against FR-24 (members-only board vs professor access) led to the read-only rule
Completeness	Is every use case in the diagram covered?	Section 9 maps all 8 use cases to requirements
Realism	Can it be built in the time left with PHP and MySQL?	Sprint planning; email reminders (FR-19) kept as Could

Check	Question we ask	How we check it
Verifiability	Can it be tested?	Every FR has a test case; every NFR has a number or a pass/fail test

Techniques used: requirements review with the professor, prototyping with UI mock-ups of the login, board, report and professor screens, and test-case generation (section 9).

10.2 Change management

1. A change is raised as a GitHub issue labelled `change-request`.
2. At sprint planning, the team checks its effect on the requirements, the design and the schedule.
3. If accepted, this document is updated (requirement, user story, traceability) and its version goes up.
4. The change is built and tested in a sprint like any other story.

10.3 Version history

Version	Date	Change
1.0	Nov 11, 2026	Full SRS after Sprint 1: process, 24 FRs, 14 NFRs, 9 user stories, 8 use cases, traceability
0.2	Nov 9, 2026	Professor feedback: state the process model and develop the requirements properly
0.1	Oct 26, 2026	Project proposal drafted: users, key features, technology and schedule